

Reinforcement Learning for HEVC/H.265 Intra-Frame Rate Control

Jun-Hao Hu, Wen-Hsiao Peng, and Chia-Hua Chung,
Department of Computer Science, National Chiao Tung University, Taiwan

Abstract—Reinforcement learning has proven effective for solving decision making problems. However, its application to modern video codecs has yet to be seen. This paper presents an early attempt to introduce reinforcement learning to HEVC/H.265 intra-frame rate control. The task is to determine a quantization parameter value for every coding tree unit in a frame, with the objective being to minimize the frame-level distortion subject to a rate constraint. We draw an analogy between the rate control problem and the reinforcement learning problem, by considering the texture complexity of coding tree units and bit balance as the environment state, the quantization parameter value as an action that an agent needs to take, and the negative distortion of the coding tree unit as an immediate reward. We train a neural network based on Q-learning to be our agent, which observes the state to evaluate the reward for each possible action. When trained on only limited sequences, the proposed model can already perform comparably with the rate control algorithm in HM-16.15.

I. INTRODUCTION

Machine learning techniques recently aroused great interest in the video coding community. One often asked question is how they could be applied to improve the performance of modern video codecs. Among the various encoding/decoding operations, the coding mode decision [1]–[5] is one widely-addressed issue in this context. Its task is usually formulated as a classification or regression problem, for which many classifiers/regressors, such as support vector machines [3], decision trees [4], or neural networks [1], [5], are applicable. However, most of these prior works consider the mode decisions for different coding units or at different stages to be independent tasks. The inherent interaction between them is neglected.

Motivated by this observation, we explore in this paper the potential of reinforcement learning (RL) for HEVC/H.265 intra-frame rate control. RL lends itself to dependent decision making problems, which are characterized by the goal of optimizing an objective/reward function that depends on inter-correlated decision making. As an example, in the intra-frame rate control, we hope to decide the value of quantization parameter (QP) for every coding tree unit (CTU) in a frame so that the frame-level distortion can be minimized while the bits consumed must match closely the pre-defined bit budget. Apparently, the QP value chosen for one CTU will affect the subsequent choices for the remaining CTUs through the rate constraint. Addressing such dependencies is the core of many video encoder control problems.

To the best of our knowledge, only little research has taken the RL approach to the rate control problem or video encoder control in general. Two most relevant prior works are [6] and

[2]. In [6], Helle *et al.* propose a complexity-aware encoder control for HEVC/H.265. Each encoding decision of a coding unit has a value composed of its rate-distortion cost and execution time, and this value is learned through their RL framework. In [2], Chung *et al.* introduce RL to the coding unit split decision in HEVC/H.265, for which several neural networks are learned through the RL framework to predict the rate-distortion benefit of split decisions at various depths.

In this paper, we formulate the intra-frame rate control problem as an RL problem. We train a neural network to determine the QP value for each CTU in a frame based on observing their texture characteristics—e.g. the luminance sample variance, the gradient magnitude, and the color number—and the bit balance during the encoding. No prior knowledge about HEVC/H.265 is needed. Experimental results show that when trained on few sequences for two target bit rates, our model can already perform comparably with the hand-designed rate control scheme adopted by HM-16.15 for most test sequences over a wide range of bit rates.

The remainder of this paper is organized as follows: Section II reviews the RL basics. Section III presents our RL formulation for the rate control problem. Section IV details our network implementation and training procedure. Section V compares the performance of our model with the baseline scheme in HM-16.15. Lastly, Section VI concludes this paper with remarks on future work.

II. REINFORCEMENT LEARNING (RL) BASICS

This section reviews the reinforcement learning basics to facilitate the understanding of our proposed scheme.

Reinforcement learning is a learning technique for solving decision making problems. As illustrated in Fig. 1, its setup consists of an agent and an environment with which the agent interacts. At each time step t , the agent takes an action a_t following a policy $\pi(a_t|s_t)$, which is a mapping from a state s_t summarizing the agent's observation about the environment to the action a_t . As an example, in the case of a stochastic policy, $\pi(a_t|s_t)$ may denote the probability of taking action a_t conditioned on the state s_t . After an action has been taken, the environment responds with an immediate reward r_{t+1} together with a new state s_{t+1} , which is produced according to a state transition probability $p(s_{t+1}|s_t, a_t)$ that characterizes the environment's dynamics. In an episodic task, the process repeats until a terminal state is reached.

The objective of reinforcement learning is to learn a policy $\pi(a_t|s_t)$ such that a high probability is placed on the sequences

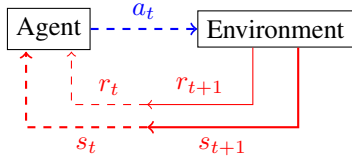


Fig. 1: Reinforcement learning setup

Algorithm 1 Q-learning

- 1: Initialize $Q(s, a)$ for all states s and actions a
 - 2: Repeat (for each episode):
 - 3: Initialize s
 - 4: Repeat (for each step of episode):
 - 5: Choose $a = \arg \max_a Q(s, a)$ with the ϵ -greedy method
 - 6: Execute a , observe the reward r and next state s'
 - 7: $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$
 - 8: $s \leftarrow s'$
 - 9: until s is terminal (α : learning rate)
-

of state-action pairs $s_0, a_0, s_1, a_1, \dots, s_t, a_t, \dots$ with high rewards. This can be achieved by maximizing the accumulated reward $G_t = \sum_{k=0}^{T-1} \gamma^k r_{t+k+1}$ starting from any time step t in expectation, where γ is a discount factor between 0 and 1 determining the importance of future rewards and T is the number of total steps, which can be infinity in continuing tasks. In symbols, we have

$$\arg \max_{\pi} E_{\pi}[G_t | s_t, a_t], \forall t. \quad (1)$$

By this, we learn a far-sighted agent, which chooses an action at every time step by taking into account its influence to the future rewards the agent will receive in the long run.

One common way for finding the optimal policy in Eq. (1) is to learn an action-value function $Q(s_t, a_t) = E_{\pi}[G_t | s_t, a_t]$ through experiences. This approach, known as Q-learning [7], avoids the need of having to know the environment dynamics $p(s_{t+1} | s_t, a_t)$. The main idea is outlined in Algorithm 1.

III. RL-BASED INTRA-FRAME RATE CONTROL

In this section, we formulate the rate control problem as a reinforcement learning problem.

We address specifically the all intra coding scenario. Each video frame is coded independently to meet a target bit budget, with the encoding of each frame considered an episodic task. Our objective is to determine the QP value for every CTU

TABLE I: State Definition

	Components
1	Y-component variance of the current CTU
2	Horizontal and vertical difference of the current CTU
3	Gradient magnitude of the current CTU
4	Color number of the current CTU
5	Average Y-component variance of the remaining CTUs
6	Average horizontal and vertical difference of the remaining CTUs
7	Average gradient magnitude of the remaining CTUs
8	Average color number of the remaining CTUs
9	Bit balance
10	Number of remaining CTUs
11	Frame-level initial QP

in a coding frame, such that the frame-level distortion can be minimized while the number of compressed bits must match the bit budget. In symbols, we have

$$\arg \min_{\{QP_i\}} \sum_i D_i(QP_i) \text{ s.t. } \sum_i R_i(QP_i) = R_c, \quad (2)$$

where $\{D_i, R_i\}$ denote respectively the distortion of the i -th CTU in mean squared error and the number of bits it consumes, and R_c is the bit budget.

In the present context, we can formulate the process of QP determination for every CTU in a frame as an episodic reinforcement learning problem. Specifically, the

- **Agent** is our CTU-adaptive rate control algorithm.
- **Actions** a 's are the delta QP values ranging from -3 to +3 relative to a frame-level initial QP.
- **State** s on which the action taking is based includes the hand-crafted features detailed in Table I.

The state summarizes not only the texture variation (components 1-4) within a coding CTU but also the spatial characteristics (components 5-8) of the remaining CTUs, the bit balance and the frame-level initial QP, which is decided by the frame-level rate control of HM-16.15. Such design allows the agent to look into the complexity of the future CTUs when making a decision for the current CTU. The state value is updated upon the completion of coding each CTU. In particular, a terminal state will be reached after coding a CTU if the bit budget is used up while there are still CTUs not yet coded in the current frame.

To achieve the goal in Eq. (2), an immediate reward r is given to the agent for each action it takes. This reward is computed to be the negative mean squared error of the CTU induced by encoding it with the chosen QP value. The negative sign accounts for our desire to reduce the distortion by increasing the reward in the learning process. In the case of a terminal state, all the remaining CTUs are encoded with QP=51 (the largest QP value possible) at training time, with the sum of their negative mean squared errors added to the current reward. Apparently, the earlier the terminal state is reached, the larger the total error, or the smaller the total reward, would be. The agent is thus discouraged to produce a sequence of QP's that has a tendency of exceeding the bit budget by a large margin. On the other hand, completing the encoding of the entire frame with a bit rate lower than the specified level would be penalized, too, presuming that a larger frame-level distortion (and thus a smaller total reward) would be resulted.

IV. IMPLEMENTATION

With the definitions of the state and actions in the previous session, we implement the action-value function $Q(s, a)$ with a neural network parameterized by θ . Due to the low complexity requirements of video applications, we adopt a 3-layer fully-connected network, with the first layer composed of 400 hidden units, the second 300 units, and the output 7 units. The input to the network is the state given in Table I.

Algorithm 2 Q-learning for the proposed model $Q(s, a; \theta)$

```

1: Initialize the network parameters  $\theta$ 
2: For Frame 0 to  $M - 1$ 
3:   Set  $R = \text{bit budget}$ 
4:   For CTU  $t = 0$  to  $N - 1$ 
5:     Set state  $s_t$ 
6:     Choose  $a_t = \arg \max_{a_t} Q(s_t, a_t; \theta)$  with  $\epsilon$ -greedy method
7:     Execute  $a_t$ 
8:      $R = R - \text{bits consumed}$ 
9:     Observe the reward  $r_{t+1}$  and next state  $s_{t+1}$ 
10:    if ( $R < 0$  and  $t \neq N - 1$ )
11:       $r_{t+1} = r_{t+1} + \text{penalty of early termination}$ 
12:      Store transition ( $s_t, a_t, r_{t+1}, s_{t+1}$ )
13:      break
14:    else
15:      Store transition ( $s_t, a_t, r_{t+1}, s_{t+1}$ )
16:  End For
17:  Sample a mini-batch of transitions from the replay buffer
18:  ( $s_{t'}, a_{t'}, r_{t'+1}, s_{t'+1}$ )
19:  Compute  $y_{t'} = r_{t'+1} + \gamma \max_{a_{t'+1}} Q(s_{t'+1}, a_{t'+1}; \theta)$ 
20:  Update  $\theta$  with the loss  $\sum_{t'} [y_{t'} - Q(s_{t'}, a_{t'}; \theta)]^2$ 
21: End For

```

For training the network parameter θ , we follow the typical Q-learning procedure, as detailed in Algorithm 2. Due to limited computational resources, the network is trained on only 5 test sequences in the common test conditions of HEVC/H.265 [8], PartyScene (Class C), BQMall (Class C), RaceHorses (Class D), KristenAndSara (Class E) and ChinaSpeed (Class F). Furthermore, only 20 frames sampled randomly from each of these sequences are used. In addition, the training is done for two target bit rates, 5 Mbps and 15 Mbps.

At test time, the encoder chooses action based on $a = \arg \max_a Q(s, a; \theta)$. If the bit budget is exceeded while there are CTUs remaining (i.e., a terminal state is reached at test time), we encode the remaining CTUs with a QP value equal to the frame-level initial QP plus 3. Note that this is different from how we handle the terminal state at training time, of which the purpose is to compute a reward signal.

V. EXPERIMENTAL RESULTS

For experiments, we implement the proposed scheme with HM-16.15 and compare its performance with the current baseline algorithm adopted by the committee software. The experiments are conducted under the All Intra configuration and with constant bit rates. The four target bit rates to be fit are determined by encoding each test sequence using fixed QP values of 22, 27, 32, and 37, respectively. In the interest of time, only the lower resolution test sequences, including both natural and screen content, in Classes C, D, E, and F are tested. The training sequences are excluded from test. The remaining settings follow the common test conditions [8], unless otherwise stated.

Table II presents the BD-rate results in percentage terms with the baseline algorithm serving as anchor. The positive (respectively, negative) numbers indicate BD-rate losses (respectively, gains). From the table, the proposed method performs comparably with the well-optimized baseline algorithm,

TABLE II: BD-rates relative to the baseline in HM-16.15

Component	Y (%)	U (%)	V (%)
Class C (WVGA)	3.8	-2.8	-1.5
Class D (WQVGA)	2.4	3.3	3.7
Class E (720p)	2.9	-2.0	-1.1
Class F (Screen content)	2.4	0.4	-0.1
Avg.	2.8	-0.3	0.3

showing only an average Y-component BD-rate loss of 2.8%. It is worth mentioning that this is achieved when the model is trained on only few sequences.

Fig. 2 further visualizes the per-frame bit rate as a function of frame index. The left and right columns correspond to the results at the highest and lowest target bit rates, respectively. We see that the baseline algorithm is relatively more accurate and stable in fitting the target bit rates, while our model performs well in most sequences at the low rate but exhibits more fluctuation at the high rate and in the screen content sequence SlideEditing. The reasons may be attributed to insufficient training. We do observe that the bit rate fluctuation reduces gradually over time during the training process. Nonetheless, more investigation is needed. It is also seen that without any rate control mechanism, the fixed QP approach has drastic bit rate fluctuation. Fig. 3 displays the corresponding Y-PSNR as a function of frame index. With the constant bit rate constraint, it is no surprise to see that both the baseline rate control algorithm and our model have apparent Y-PSNR fluctuation. This can be mitigated by allowing variable bit rate, which is in the scope of our future work.

Table III quantifies explicitly the bit rate error and fluctuation in percentage terms. The former is computed as the absolute deviation of the actual bit rate from the target at sequence level in percentage terms, while the latter measures the average per-frame bit rate deviation from the target (also in percentage). Overall, the proposed method has a small bit rate error across the entire bit rate range, although most of these rate points are not seen at training time. As found previously in Fig. 2, Table III confirms again that our model has larger bit rate fluctuation at higher bit rates.

Fig. 4 compares side-by-side the reconstructed frames produced by the baseline algorithm and our scheme. Additionally, we visualize the QP values of CTUs in heat maps. Observe the discrepancy between the heat maps. In this test, the proposed method leads to subjectively more pleasing quality, showing more texture details.

VI. CONCLUSION AND FUTURE WORK

In this paper, we explore the applicability of reinforcement learning to video encoder control. Specifically, we address the problem of HEVC/H.265 intra-frame rate control. A neural network is trained to determine the quantization parameter for every coding tree unit in a frame subject to a rate constraint. The model need not know the inner workings of HEVC/H.265. Experiments show promising results. We believe it deserves more effort to understand the full potential of reinforcement learning in the area of video coding.

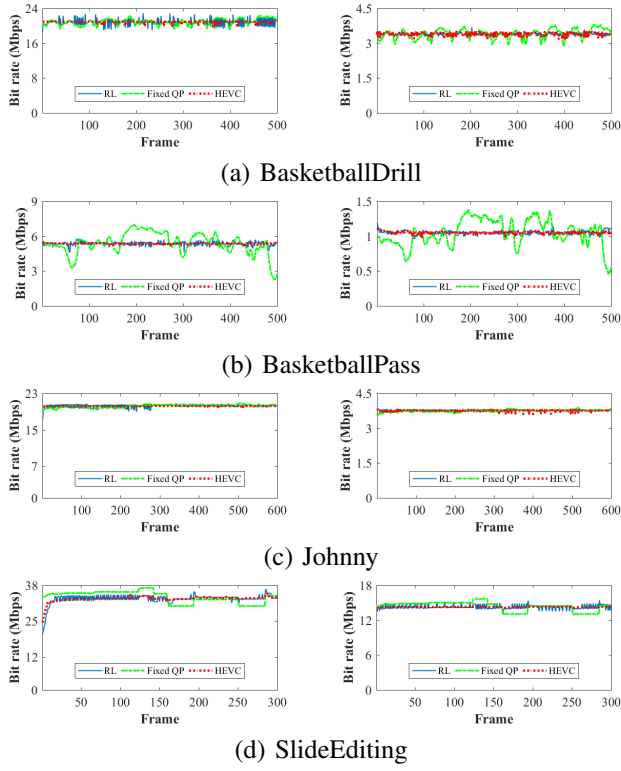


Fig. 2: Bit rate as a function of frame index: high rate (left) and low rate (right).

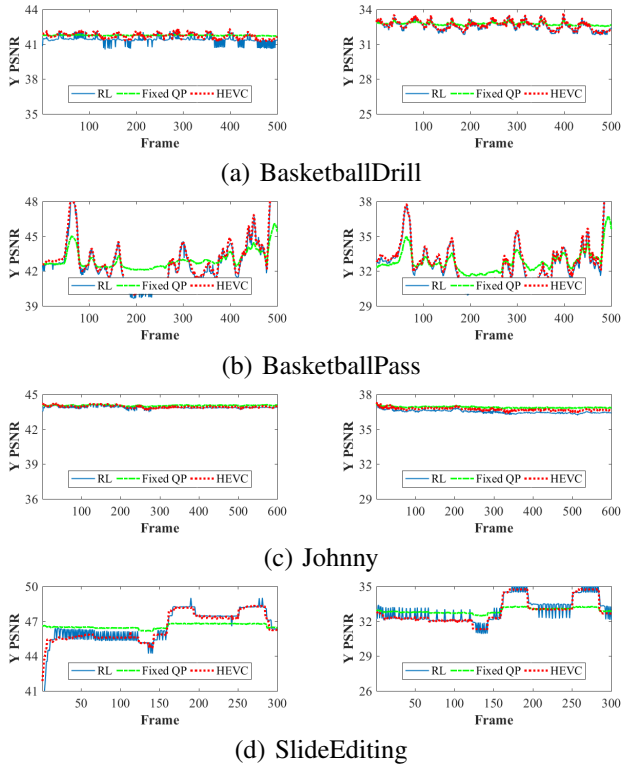


Fig. 3: Y-PSNR as a function of frame index: high rate (left) and low rate (right).

TABLE III: Bit rate error and fluctuation (%)

Sequences	Bit rate Kbps	HM-16.15		RL	
		Err. (%)	Var. (%)	Err. (%)	Var. (%)
BasketballDrill	20854	0.08	0.54	0.05	2.62
	11346	0.12	1.01	0.07	2.45
	6105	0.16	1.60	0.13	1.98
	3385	0.23	1.77	0.20	1.21
BasketballPass	5355	0.18	0.60	0.06	2.34
	3199	0.33	0.64	0.29	2.33
	1847	0.52	0.85	0.58	1.99
	1042	0.81	1.19	0.92	1.98
Johnny	20191	0.02	0.14	0.02	1.30
	11277	0.07	0.14	0.02	0.59
	6469	0.13	0.18	0.12	0.77
	3752	0.22	0.48	0.23	0.67
SlideEditing	33449	0.93	1.36	0.87	2.90
	25005	0.64	0.87	0.51	2.58
	19075	0.34	0.56	0.21	2.35
	14271	0.04	0.30	0.00	2.32

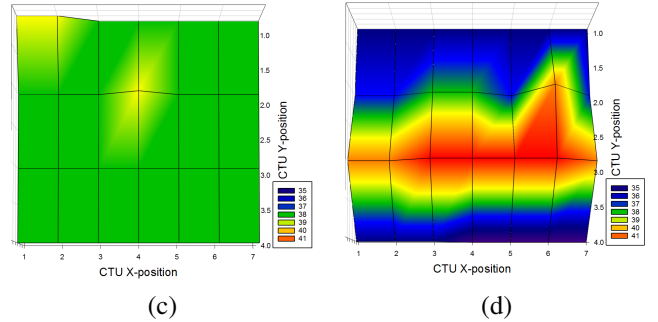


Fig. 4: Reconstructed frames and heat maps of QP values for BasketballPass at 1042 Kbps: (a)(c) HM-16.15, and (b)(d) the proposed method.

REFERENCES

- [1] T. Laude and J. Ostermann, "Deep learning-based intra prediction mode decision for HEVC," *Picture Coding Symp. (PCS)*, Nuremberg, 2016, pp. 1-5.
- [2] C. H. Chung, W. H. Peng and J. H. Hu, "HEVC/H.265 coding unit split decision using deep reinforcement learning," in *Proc. IEEE Int. Symp. Intell. Signal Process. and Commun. Syst. (ISPACS)*, Xiaman, 2017.
- [3] X. Shen and L. Yu, "CU splitting early termination based on weighted SVM," *EURASIP J. on Image and Video Process.*, no. 1, pp. 111, Jan. 2013.
- [4] G. Correa, P. Assuncao, L. Agostini and L. da Silva Cruz, "Classification-based early termination for coding tree structure decision in HEVC," in *Proc. IEEE Int. Conf. Electron., Circuits, and Syst. (ICECS)*, Marseille, Dec. 2014, pp.239-242.
- [5] F. Mu, L. Song, X. Yang and Z. Luo, "Fast coding unit depth decision for HEVC," in *Proc. IEEE Int. Conf. Multimedia and Expo Workshops (ICMEW)*, Chengdu, 2014, pp. 1-6.
- [6] P. Helle, H. Schwarz, T. Wiegand and K. R. Miller, "Reinforcement learning for video encoder control in HEVC," in *Proc. Int. Conf. Syst., Signals and Image Process. (IWSSIP)*, Poznan, 2017, pp. 1-5.
- [7] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Mach. Learn.*, vol. 8, no. 3-4, pp. 279-292, 1992.
- [8] K. Sharman and K. Shring, "Common test conditions for HM," *ITU-T SG16 WP3 and ISO/IEC JTC1/SC29/WG11, JCTVC-X1100*, Jun. 2016.